
INFORME DE PRÁCTICA 2

Calibración de Cámaras y ojo de pez

Visión por Ordenador

AUTORES

Miguel Sobral Barrento

Alonso Bernabé Sanjuan

3º INGENIERÍA MATEMÁTICA E INTELIGENCIA ARTIFICIAL

Curso 2026/27

1 · INICIO

Objetivo y preparación

La práctica estima los parámetros intrínsecos y extrínsecos de dos cámaras estéreo y calibra una cámara ojo de pez para corregir su distorsión. Se trabaja con tableros de ajedrez y OpenCV 4.8.0.

2 · DESARROLLO

Apartado A · Calibración de las cámaras

Cámara izquierda

Tarea A.1. `load_images()` carga las 19 imágenes de la carpeta left.

Tarea A.2. `findChessboardCorners()` busca el patrón de 8×6 esquinas. `cornerSubPix()` las refina en escala de grises con ventana 5×5 y criterio de 30 iteraciones o $\epsilon = 0.01$; se detecta el patrón en 18 imágenes.

Tarea A.3. `drawChessboardCorners()` se utiliza para comprobar las detecciones iniciales. La figura muestra una de las vistas válidas.

Tarea A.4. `get_chessboard_points()` genera 48 puntos coplanares separados 30 mm y conserva el orden de detección. X recorre los 5 intervalos del lado corto (150 mm) e Y los 7 del lado largo (210 mm), como indica el enunciado.

Tarea A.5. `calibrateCamera()` estima los parámetros usando solo las detecciones válidas.

```
K (px) = [[419.1737, 0.0000, 149.8720]; [0.0000, 420.6753, 128.8857]; [0.0000, 0.0000, 1.0000]]
D = [0.02547686, -3.34669231, 0.00493017, -0.00684827, 20.02812522] [k1, k2, p1, p2, k3]
Error RMS = 0.153073113716 px
```

3 · PREGUNTAS

Pregunta A.1 · Cámara derecha

Se repite el proceso con las 19 imágenes de right. Se detectan 18 patrones de 48 esquinas; las figuras son salidas individuales de `cv2.drawChessboardCorners()`.

```
K (px) = [[432.1918, 0.0000, 146.7782]; [0.0000, 435.4478, 137.2778]; [0.0000, 0.0000, 1.0000]]
D = [-0.11862371, -0.18601075, 0.00625041, -0.00822418, 1.08945344] [k1, k2, p1, p2, k3]
Error RMS = 0.205888506209 px
```

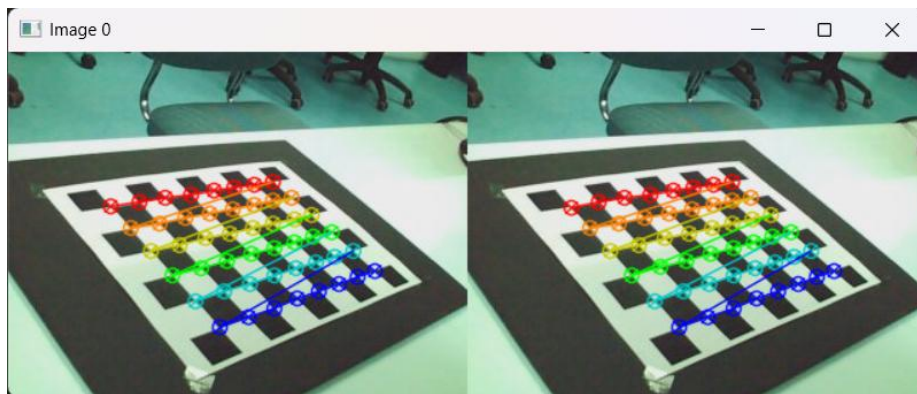
La tabla recoge los vectores `rvec` y `tvec` devueltos por `calibrateCamera()` para cada una de las 18 vistas válidas, en el orden de procesamiento.

Poses de la cámara derecha

Imagen	rvec (rad)	tvec (mm)
right_003	[1.9393, 1.1764, -0.0648]	[-122.62, -71.64, 508.59]
right_011	[2.0077, 1.5049, -0.0049]	[-164.50, -83.07, 575.83]
right_008	[1.9658, 1.3957, 0.0104]	[-148.66, -101.94, 522.96]
right_005	[1.9507, 1.2379, -0.0714]	[-127.43, -78.19, 523.89]
right_002	[1.9204, 1.1322, -0.0644]	[-129.55, -60.93, 508.02]
right_018	[2.1310, 1.7407, -0.0859]	[-134.70, -95.01, 646.92]
right_007	[1.9690, 1.3734, -0.0272]	[-132.06, -99.36, 521.11]
right_010	[2.0050, 1.4420, -0.0063]	[-156.58, -94.45, 545.52]
right_004	[1.9456, 1.2034, -0.0742]	[-121.39, -76.93, 515.74]
right_015	[2.0856, 1.7124, 0.0067]	[-170.73, -109.87, 611.13]
right_009	[1.9746, 1.4188, 0.0281]	[-162.86, -106.61, 527.97]
right_017	[2.1215, 1.7348, -0.0783]	[-144.94, -91.94, 641.41]
right_001	[1.8948, 1.0966, -0.0888]	[-118.12, -45.90, 518.40]
right_013	[2.0377, 1.5694, 0.0171]	[-194.92, -72.29, 600.62]
right_006	[1.9523, 1.3074, -0.0461]	[-135.03, -81.41, 518.27]
right_012	[2.0227, 1.5199, -0.0108]	[-176.28, -71.93, 592.93]
right_000	[1.8738, 1.0657, -0.1267]	[-91.69, -36.19, 526.70]
right_016	[2.1123, 1.7364, -0.0271]	[-162.06, -100.47, 625.86]

Pregunta A.2 · Refinamiento subpíxel

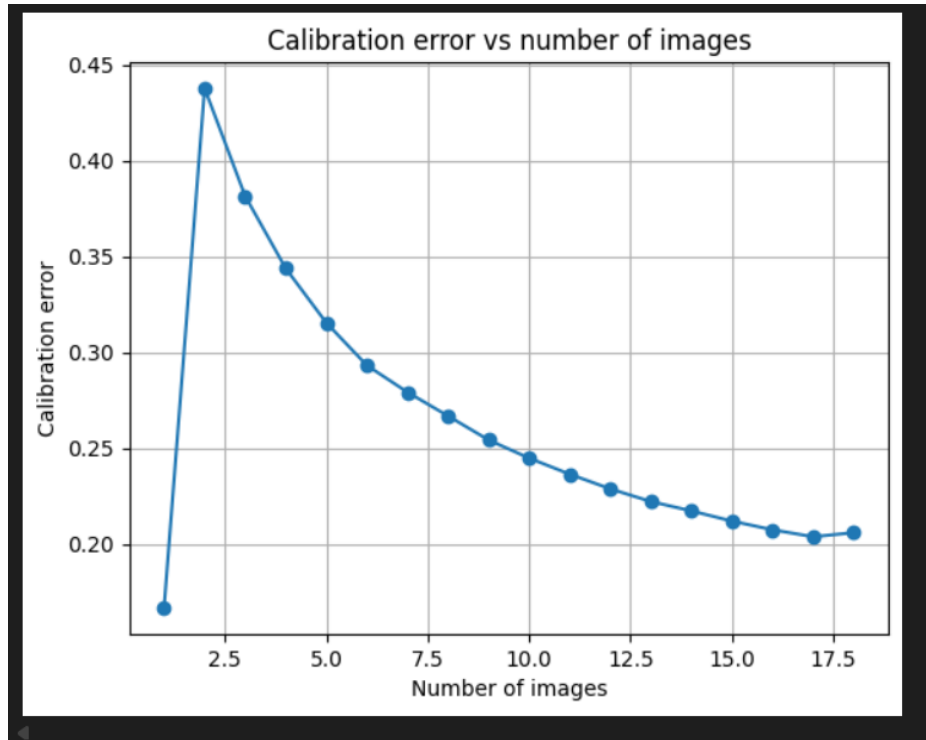
La comparación de las coordenadas antes y después de `cornerSubPix()` da un desplazamiento medio de 0.113 px por vista, con valores entre 0.065 y 0.478 px. En 16 de las 18 vistas el cambio es inferior a 0.11 px; la imagen adjunta muestra la entrada y la detección refinada.



Salida de comparación de `cv2.imshow()`: detección inicial a la izquierda y refinada con `cornerSubPix()` a la derecha.

Pregunta A.3 · Número de imágenes

Con 18 vistas, el error es de 0.206 px; entre 15 y 18 vistas pasa de 0.216 a 0.206 px. Para el proyecto final conviene tomar del orden de 10^1 imágenes (unas 15–20) y variar la posición y orientación del tablero.



Error de calibración frente al número de imágenes, según la salida del notebook.

4 · DESARROLLO

Apartado B · Corrección de distorsión ojo de pez

Calibración

Tarea B.1. `load_images()` carga las 10 imágenes de la carpeta fisheye.

Tarea B.2. `findChessboardCorners()` busca un patrón de 7×6 ; se obtienen 10 detecciones válidas. Se convierten a escala de grises y se refinan con `cornerSubPix()`, ventana 5×5 y criterio de 30 iteraciones o $\epsilon = 0.1$.

Tarea B.3. `get_chessboard_points()` genera 42 puntos para el patrón de 7×6 , con separación de 30 mm y forma (n, 1, 3). X recorre el lado corto (150 mm) e Y el largo (180 mm).

Tarea B.4. Se inicializan K y D a cero, y se crean vectores de rotación y traslación por vista; las flags son `CALIB_RECOMPUTE_EXTRINSIC` y `CALIB_FIX_SKEW`.

Tarea B.5. `fisheye.calibrate()` estima K, los cuatro coeficientes de distorsión y el error RMS.

```
K (px) = [[188.6232, 0.0000, 503.9163]; [0.0000, 185.0892, 373.6582]; [0.0000, 0.0000, 1.0000]]
D = [0.08040023, 0.00739159, 0.03085157, -0.03012552] [k1, k2, k3, k4]
Error RMS = 141.676796042 px
```

El RMS de 141.677 px sigue siendo muy superior al de las cámaras estéreo, por lo que la corrección ojo de pez debe interpretarse con cautela.

5 · PREGUNTAS

Pregunta B.1 · Imágenes sin distorsión

Con la matriz K y el vector D mostrados en la Tarea B.5 se construyen los mapas y se remuestrean las dos primeras imágenes con interpolación lineal. Se muestra cada salida corregida por separado, como en el código.



Salida corregida de `cv2.remap()` para `VMRImage0.jpg`.



Salida corregida de `cv2.remap()` para `VMRImage1.jpg`.

6 · CIERRE

Conclusiones

Las cámaras izquierda y derecha presentan errores RMS de 0.1531 y 0.2059 px. El error de 141.677 px en la calibración ojo de pez sigue siendo alto; antes de usar la corrección de forma cuantitativa habría que revisar ese ajuste.